

14. Notification System

Interviewer: Design a notification system. Any internal service — orders, payments, marketing — should be able to say "tell this user something," and we deliver it over **push, SMS, and email**. Go.

Candidate: Good one — this isn't really about a clever algorithm, it's about **decoupling a fast, bursty producer from slow, unreliable third-party providers**, while never double-notifying a user. Let me clarify the shape first.

1. Clarifying the requirements

Candidate: A few questions: 1. Which channels, exactly — push (APNs/FCM), SMS (Twilio-style), email (SES-style)? Does one logical event fan out to multiple channels, or does the caller pick one? 2. Do users control this — opt-in/opt-out per channel, quiet hours? 3. Who owns the copy — do product/marketing edit templates without a code deploy? 4. What happens if a producer retries the same event — do we guarantee no duplicate send? 5. Is this fire-and-forget, or does the caller need to know delivery actually succeeded?

Interviewer: All three channels, and yes — one event like "order shipped" can fan out to push *and* email. Users have per-channel preferences. Marketing edits templates through a dashboard. Producers *will* retry on timeout, so duplicates must be prevented. And yes, we need delivery tracking — sent, delivered, failed, bounced — queryable later.

Candidate: Then here's what I'm building:

Functional - `send(userId, eventType, channels[], templateId, data)` — any internal service can call this. - Fan-out: one event → up to 3 channel sends (push, SMS, email). - Per-user, per-channel opt-in/opt-out and quiet hours, respected at send time. - Template management: edit copy without redeploying the notification service. - **Deduplication**: a retried or replayed event must not double-notify. - **Rate limiting** per user (a bug shouldn't fire 50 pushes in a minute). - **Delivery tracking**: status per notification, per channel.

Non-functional - **Producers must never block** on a slow SMS/email provider — this is the core constraint. - **At-least-once delivery** — dropping a notification silently is worse than an occasional retry, so duplicates are an accepted side effect that we must then suppress ourselves. - **Bursty**: a marketing campaign or an incident page can spike volume 50–100x instantly. - **Partial failure is fine**: if SMS fails but email succeeds for the same event, that's a correct outcome, not a bug.

2. Estimation

Candidate: Let me size it so the queue/partition decisions aren't arbitrary.

Assume 50M users, ~5 notifications/user/day across all channels.

events/day = 250M

events/sec = 250M / 86,400 $\approx$ 2,900/sec average

Fan-out: one event $\rightarrow$ up to 3 channel-sends

channel-sends/day $\approx$ 750M $\rightarrow$ ~8,700/sec average, peak (campaign) $\approx$ 100k+/sec

Delivery log: 750M rows/day * ~200 bytes $\approx$ 150 GB/day

$\rightarrow$ this alone rules out a single relational table; needs a wide-column/KV store that scales writes horizontally.

Queue at peak: 100k msgs/sec $\rightarrow$ needs dozens of partitions (Kafka) or many parallel queues (SQS), NOT a single FIFO queue.

$\Rightarrow$ The real numbers that shape this design: 100x bursts, 750M/day append-only writes, and a hard requirement that the producer's POST never waits for any of this.

3. A naive V1 (and why it breaks)

Candidate: Let me state the naive version so we're aligned on why it can't ship.

```
[ Order Service ] --POST /notify--> [ Notification Service ] --HTTP--> [ Twilio ]
                                     --HTTP--> [ SES ]
                                     --HTTP--> [ APNs ]
                                     (all synchronous, in the request path)
```

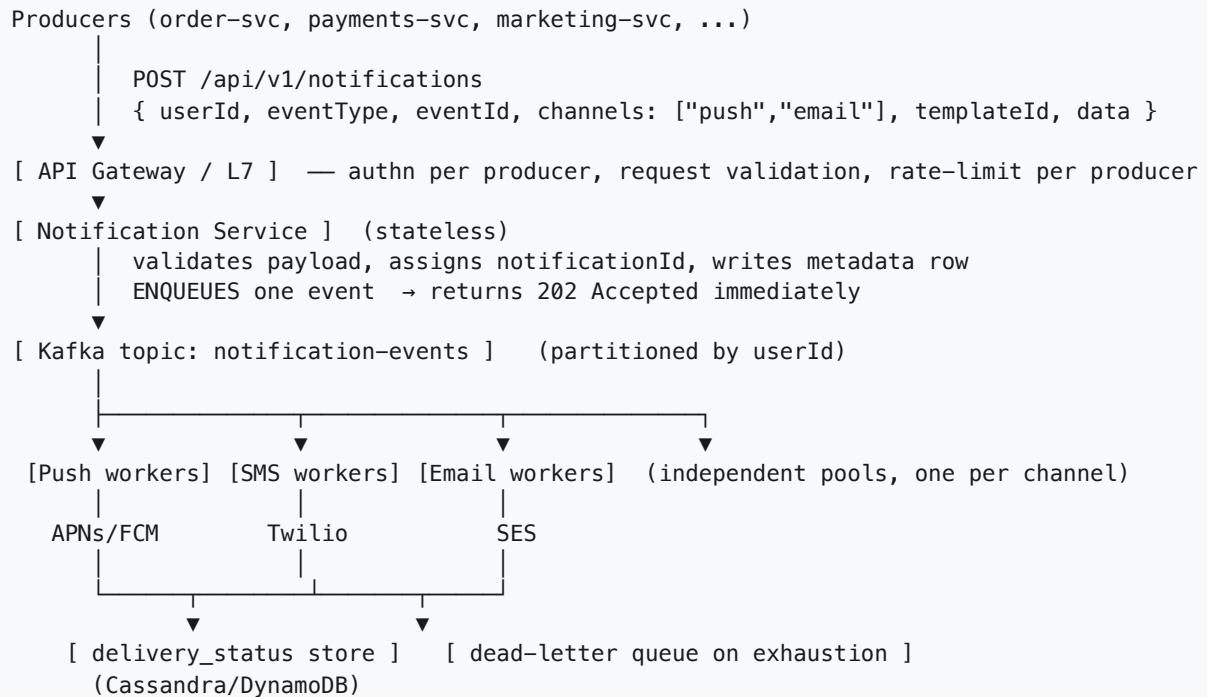
Why it dies: the caller's request now takes as long as the **slowest** provider call — a Twilio bad day (2s $\rightarrow$ 20s per call) slows down every order-shipped event in the system. No queue means no buffering for the 100x campaign spike — we'd hammer three providers at once and get rate-limited or blacklisted. A crash mid-fan-out (push sent, email not yet called) leaves no record and no way to resume. And there's no dedup anywhere — a producer's retry after a timeout sends the SMS twice.

Candidate: So the fix isn't "add more servers," it's "**put a durable queue between the producer and every provider call**, and make the provider calls fully async." That single move is what makes this problem tractable.

4. V1, done right — the async pipeline

Interviewer: Okay, show me the queue-based version.

Candidate:



- **202 Accepted, before any provider is called.** This is the signal that the whole system is async — the caller's job ends at "I persisted your intent to notify," not "I delivered it."
- **One event, fanned out to N channel-workers.** A worker per channel because APNs, Twilio, and SES have wildly different throughput/latency/rate-limit profiles — coupling them into one worker pool means the slowest provider throttles all three.
- **Kafka partitioned by `userId`** keeps ordering per user (useful, though not strictly required here) and lets us scale consumers horizontally by adding partitions/workers.

5. Scaling the write path — the 100x campaign spike

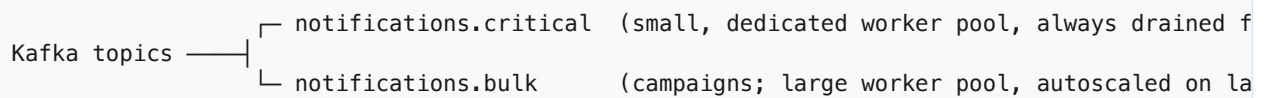
Interviewer: Marketing hits "send" on a campaign to 20M users at once. Walk me through what happens to your pipeline.

Candidate: This is exactly the burst the queue exists for.

- The notification service **enqueues fast** — writing to Kafka is cheap (sequential disk append), so ingest absorbs the 20M-event burst in seconds without ever calling a provider.
- **Workers drain at the rate the provider can sustain**, not the rate producers can push. Each channel's worker pool is **sized to that provider's rate limit** (e.g., Twilio might cap us at 500 SMS/sec per account) — going faster just gets us throttled/banned, so the queue's backlog is the correct, intentional buffer.
- **Priority separation:** a security alert ("your password changed") must not sit behind 20M marketing emails. I'd use **separate topics/queues by priority** — `notifications.critical` and

`notifications.bulk` — with workers configured to always drain `critical` first (or simply give `critical` its own dedicated worker pool so it's never contended).

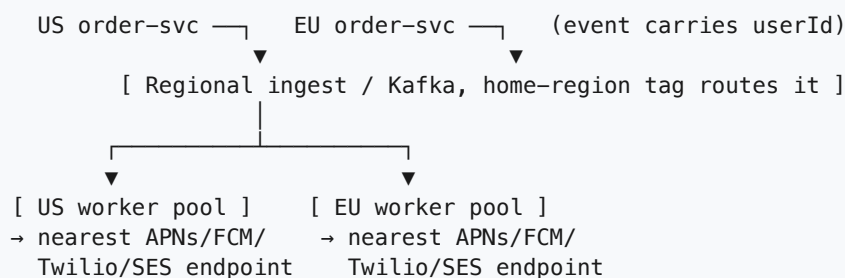
- **Autoscaling workers** on queue depth (a standard consumer-lag-based autoscaler) so a 10x spike in backlog spins up more consumers automatically.



6. Multi-region

Interviewer: We're global now — users in the US and EU. What changes?

Candidate: Two separate concerns: **where the event is produced** vs. **where the user lives** — and I route by the latter.



Deliver from the **worker pool nearest the user**, not nearest the producer — a US service notifying an EU user should still send via the EU-region endpoint (lower latency, and keeps EU user data processing in the EU if residency rules require it). **Templates/preferences replicate to all regions** (small, read-mostly); **delivery status stays regional** — it's rarely read cross-region, so there's no reason to pay replication cost on 150 GB/day of logs.

7. Latency — where the ms go

Interviewer: A user pays for an order. How fast does their "order confirmed" push arrive, and where's the time actually spent?

Candidate: Split it into two very different latencies:

- **Producer-facing latency** (the POST call) — tens of ms: validate, write a small metadata row, publish to Kafka, return 202. No provider call is in this path, so it's essentially "how fast can I append to a log."
- **End-to-end delivery latency** (event → phone buzzing) — queue-wait + worker-processing + provider latency. Under normal load queue-wait is near-zero; under a campaign spike it becomes

the dominant term, which is an accepted tradeoff — bulk notifications degrade in latency, critical ones don't (hence separate queues).

- Inside a worker, preference lookup + template render are **cache hits** (sub-ms), so the provider call itself (tens to hundreds of ms) dominates — provider latency we don't control, which is exactly why it must never sit on the producer's critical path.

8. Consistency — at-least-once delivery, and deduplication

Interviewer: You said "at-least-once." That means duplicates are possible. How do you stop a user getting the same SMS twice — this is the crux of the problem for me.

Candidate: Right — two independent sources of duplication, and I need to guard both:

1. **Producer retries.** If the producer's POST times out (maybe we *did* enqueue, but the ack was lost), it retries with the same logical event. I ask producers to pass an `eventId` they control (e.g. `order_shipped:order_123`), and I compute: `dedup_key = hash(userId + eventType + eventId)` On ingest, `SET dedup:{dedup_key} NX EX 24h` in Redis. If it already exists, this is a producer retry of an already-accepted event → return the **same** `notificationId` (idempotent response), don't enqueue a second event.
2. **Queue redelivery.** Kafka/SQS guarantee at-least-once, so a worker crash mid-processing means the same message is redelivered. Before calling the provider, the worker does: `SET dedup:{dedup_key}:{channel} NX EX 24h` (atomic, per channel) → already set ⇒ we already attempted/sent this channel; skip, log as suppressed → newly set ⇒ proceed to call the provider The dedup key is **per (user, event, channel)** deliberately — a user can legitimately get *both* a push and an email for the same event, just not two pushes for it.

Worker receives message (possibly a redelivery)

```
├ SET dedup:{userId}:{eventId}:{channel} NX EX 24h
│   ├── already exists → skip send, mark delivery_status=duplicate-suppressed
│   └── newly set ↓
├ check user_preferences: opted in for {channel}? quiet hours?
│   ├── no → mark delivery_status=skipped-by-preference, stop
│   └── yes ↓
├ render template, call provider
└ write delivery_status (sent | failed), ALWAYS — even on failure
```

Why Redis `SET NX` and not a DB unique constraint? At 100k+ channel-sends/sec during a spike, a Postgres `UNIQUE` check becomes contended write traffic on one table. Redis gives an atomic, in-memory check-and-set that's naturally sharded by key — no single hot row.

What's strong vs. eventual here: the dedup check is the one thing that must be atomic and immediate (race two workers processing the same redelivered message, and the `SET NX` decides exactly one wins). Everything downstream — the `delivery_status` row, the "delivered" webhook update — is eventually consistent; a status update landing 200ms late harms nobody.

9. Async — this is the whole point of the design

Interviewer: You've leaned on "async" a lot already. Make it explicit — what exactly moves off the critical path, and what's the actual mechanism?

Candidate: Everything after "I accepted your request to notify" is async: the **provider HTTP call** (APNs/Twilio/SES) happens in a worker, not the producer's thread, so a 20-second Twilio outage delays SMS delivery, not order processing; the **delivery_status write** happens after the provider responds; the **retry loop** on failure is entirely worker-side. The **only synchronous part**, from the producer's point of view, is validate → dedup-check → append to Kafka → return 202 — everything else is worker-side and async.

Why Kafka specifically: partitioned, durable, replayable log. If a worker pool crashes, nothing is lost — we resume from the last committed offset; if we need to re-drive a whole campaign after a bug, we replay the partition, and the retained log doubles as the audit trail the delivery-tracking requirement basically demands anyway. **Why not just SQS?** It's a reasonable alternative for pure fire-and-forget queueing — I'd consider it if we didn't need per-user ordering or replay-for-audit — but the architectural shape (async, worker-drained, per-channel pools) doesn't change either way.

10. Caching — templates and preferences on the hot path

Interviewer: Every single worker, for every message, needs the user's preferences and the template body. That's your highest-QPS read. How do you keep it cheap?

Candidate: Cache-aside, in front of two very different backing stores:

Worker processing a message:

1. GET pref:{userId}:{channel} (Redis) — HIT: use it. MISS: read Postgres, cache ~5min TT if opted out or in quiet hours → stop here, never call the provider
2. GET template:{templateId} (Redis) — near-static, LONG TTL, invalidate on edit
3. render template with payload data
4. call provider
5. write delivery_status — always, success or failure

Templates are near-static (marketing edits rarely) → long TTL plus explicit invalidation on save, so an edit takes effect immediately. **Preferences** change more often (opt-out any time) → shorter TTL — bounds how stale an opt-out can be (more in follow-ups). **Why cache at all, given this is already async?** At 100k channel-sends/sec, a worker hitting Postgres for every lookup would need an enormous connection pool and fall over during a campaign; caching keeps the relational store's load flat regardless of notification volume.

11. Data storage — the split

Interviewer: One database for all of this, or several?

Candidate: Split strictly by access pattern — this is not a place for one-size-fits-all.

templates, user_preferences → Postgres (or a document store)
– low volume, low write rate, benefits from structure + strong consistency (a botched preference write is a real user complaint).

delivery_status / delivery log → Cassandra / DynamoDB (wide-column/KV)
– 750M rows/day, append-mostly, always looked up by notification_id or userId+time → no joins needed, needs to scale writes horizontally. A relational table buckles here and gains nothing from relations it never uses.

12. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Event backbone / queue	Kafka , partitioned by <code>userId</code>	Durable, replayable log that decouples bursty producers from slow providers; doubles as an audit trail; partitioning gives per-user ordering and horizontal worker scaling. <i>Not calling providers synchronously</i> — a slow Twilio call would then block the producer's original request, which is the one thing we must never do. SQS is a valid alternative for pure fire-and-forget queueing, but we lose easy replay/audit without extra plumbing.
Dedup + rate limiting	Redis , atomic SET NX + sliding-window INCR	O(1) in-memory check-and-set, naturally sharded by key, handles 100k+ ops/sec per shard. <i>Not a DB unique constraint</i> — that pushes uniqueness-checking write contention onto a relational table under bursty load.
Delivery status / log	Cassandra or DynamoDB (wide-column/KV)	750M writes/day, append-mostly, always keyed by <code>notification_id</code> or <code>userId+time</code> — exactly the KV/wide-column sweet spot; scales writes horizontally by adding nodes. <i>Not Postgres</i> — a single relational table would need constant re-sharding and gains nothing from joins we never do here.
Templates + preferences	Postgres (or a document store)	Low volume, needs structure (versioned templates) and strong consistency for a user's opt-out to reliably stick. <i>Not the same KV store as delivery logs</i> — mixing a tiny, structured, consistency-sensitive dataset with a firehose of append-only logs forces a bad compromise for both.
Provider calls	Per-channel worker pools (push / SMS / email each independent)	Each provider (APNs, Twilio, SES) has a different rate limit and latency profile; independent pools mean a slow/throttled Twilio never starves push or email workers of capacity.

Concern	Choice	Why this, and why not the alternative
Why not synchronous end-to-end	Async 202-Accepted pipeline	The producer's request must never wait on a third party we don't control; a provider outage would otherwise cascade into every service that sends notifications.
Ingest edge	L7 API Gateway	Auth per producer service, payload validation, and per-producer rate limiting <i>before</i> anything reaches the queue — so one noisy/buggy producer can't flood Kafka for everyone else.

The one-sentence justification you'd say out loud: "A durable, partitioned queue is the backbone that lets fast, bursty producers hand off work with a 202 and move on, while independent per-channel worker pools drain at whatever rate each slow third-party provider can actually sustain; Redis gives me the atomic dedup and rate-limit checks that at-least-once delivery demands; and I split storage between a small, consistency-sensitive relational store for templates/preferences and a horizontally-scaling wide-column store for the hundreds of millions of delivery-log rows produced every day."

13. Failure modes — provider down, retry storms, and graceful degradation

Interviewer: Twilio goes down for 20 minutes during your campaign. What actually happens across the system, end to end?

Candidate: This is where the retry policy matters — naive retries make it worse, not better.

```

Worker calls Twilio -> times out / 5xx
    |
    v
Exponential backoff with jitter: 1s, 2s, 4s, 8s, ... capped at e.g. 60s
    (jitter avoids every retrying worker syncing up and
    hammering Twilio at the exact same instant - a
    "retry storm" that keeps the outage alive)
    |
    v
Retry count exceeds N (e.g. 5)?
    |
    +-- NO -> back into the retry loop, message stays "in flight"
    |
    +-- YES -> move to Dead Letter Queue (DLQ)
                - delivery_status = failed, provider_error logged
                - alert fires on DLQ depth
                - once Twilio recovers, a re-drive job replays the DLQ
                  (never silently dropped)
  
```

Failure	What happens	Mitigation
One provider (Twilio) down	SMS backs up in its worker pool / DLQ	Push and email are unaffected — independent pools; exponential backoff + jitter avoids a retry storm; DLQ + re-drive once it recovers
Queue backlog spikes (campaign)	Delivery delayed, not lost	Autoscale workers on consumer lag; critical-priority topic drains first; bulk notifications degrade gracefully in latency, not correctness

Failure	What happens	Mitigation
Worker crashes mid-send	Message redelivered (at-least-once)	Dedup key (section 8) guarantees the user isn't double-notified even though the message is reprocessed
Notification service itself down	Producers' POSTs fail fast	Producers retry with their own backoff; nothing was enqueued, so nothing is silently lost — the failure is visible, not silent
Redis (dedup/prefs cache) down	Risk of duplicate sends or stale prefs	Redis is clustered/replicated; on a cache miss for prefs, fall back to the DB read (slower, still correct); dedup keys have a 24h TTL so a brief Redis blip is a bounded window, not a permanent hole
Delivery-log store (Cassandra) slow	delivery_status writes lag	Writes are already off the user-facing path; a status update landing late doesn't block or fail the send itself

Design principle I'd say out loud: the queue absorbs the mismatch between fast, bursty producers and slow, sometimes-down providers — that decoupling, plus backoff-with-jitter and a DLQ instead of a silent drop, is what keeps one provider's bad day from becoming an outage for the whole system.

14. Follow-up curveballs

Interviewer: A user opts out of SMS *while* their notification is sitting in the queue. Do we still send it?

Candidate: The preference check happens **at send time, in the worker, not at enqueue time**. The event just carries "send SMS for this user"; the worker looks up the live (cached, short-TTL) preference right before calling Twilio. If they opted out in the last few minutes and the cache hasn't caught up yet, worst case is one notification lands slightly after opt-out — acceptable, and the TTL is tuned specifically to bound that window.

Interviewer: How do you know a "sent" SMS was actually *delivered*, not just accepted by Twilio?

Candidate: "Sent" and "delivered" are different events. Twilio/SES/APNs all support **delivery webhooks/callbacks** — a separate lightweight endpoint receives `delivered` / `bounced` / `opened` events asynchronously and updates the same `delivery_status` row by `notification_id`. The worker's job ends at "provider accepted it"; the callback closes the loop later, independently.

Interviewer: A user gets 15 "someone commented" events in 2 minutes. Do they get 15 pushes?

Candidate: They shouldn't — that's a UX problem the queue alone doesn't solve. I'd add a short **debounce/aggregation window** in the notification service for certain event types: hold similar low-priority events for e.g. 60 seconds and collapse them into one "3 new comments" digest before enqueueing the actual send. Combined with the per-user rate limiter (`INCR user:{id}:count EX 60` in Redis, capping sends per minute and dropping/delaying the excess for non-critical types), this keeps a chatty event source from becoming a spam problem.

Interviewer: Can a critical security alert ever get stuck behind a marketing campaign?

Candidate: Not with the design as built — critical and bulk notifications live on separate Kafka topics with **separate, dedicated worker pools**. Even if `notifications.bulk` has a backlog of 20 million messages, `notifications.critical` has its own consumers that were never contending for that capacity. The one thing I'd explicitly test is that autoscaling policy and worker-pool sizing are set so the critical pool is never starved of compute during a bulk spike.

15. Deep-dive: "Why not just call the provider inline, in the request that triggers it?"

Interviewer: Section 3 waved at this, but let's slow down. Why not skip Kafka entirely? When `order-svc` ships an order, have it call Twilio/SES/APNs **directly, in that same request**. One fewer moving part, one fewer thing to operate. Convince me it's wrong.

Candidate: It's the first design everyone reaches for, and it's wrong for four compounding reasons — not just "it's slow," but a chain of failures that gets worse under load, which is exactly when you can least afford it.

1. Coupling your uptime to a third party you don't control. The instant `order-svc` calls Twilio inline, `order-svc`'s availability becomes `min(order-svc, Twilio)`. A Twilio brownout — not even a full outage, just degraded latency — now degrades *order placement*, a completely unrelated capability, because the two are welded together in one request.

2. Request latency inherits the slowest provider. APNs, Twilio, and SES don't have the same latency profile, and none of them are "fast" in the sense a caller needs. If one event fans out to three channels sequentially inline, the caller waits for `push + sms + email`, not `max(push, sms, email)` — worse, if any provider is doing a 10s retry-with-backoff internally before failing, the caller's HTTP request is now open for 10+ seconds.

3. Retry storms. The caller's HTTP client has its own timeout and retry policy (say, 3s timeout, retry twice). If the provider is merely *slow* rather than *down*, the caller times out and retries — but the first call may still complete server-side. Now every producer's retry policy is layered on top of the provider's own flakiness, with **no queue to absorb it**: load on the already-struggling provider goes *up* exactly when it should go down, and it goes up multiplicatively across every producer service doing this independently. This is the textbook retry-storm feedback loop.

4. Loss on crash, with no record. If `order-svc`'s process dies after calling Twilio but before returning to its own caller — or dies partway through a 3-channel fan-out, having sent the push but not yet called SES — there is **no durable trace** that a notification was attempted. Nothing to replay, nothing to reconcile, nothing to alert on. It's silently gone, and worse, we don't know it's gone.

SYNCHRONOUS INLINE:

order-svc → HTTP → Twilio (slow/down) → order-svc's own request now blocked
|
└ order-svc crashes here → notification vanishes, no record, no retry, no alert

ASYNCHRONOUS QUEUE + WORKERS:

order-svc → enqueue (ms) → Kafka (durable) → worker → HTTP → Twilio (slow/down)
|
returns 202 immediately
|
message stays put, replayed on worker crash
Twilio's bad day only delays SMS workers, nothing else

Comparison

Concern	Synchronous inline send	Async queue + workers
Producer latency	= sum/max of every provider called, inline	tens of ms — validate + enqueue, provider never in the request path
Blast radius of a provider outage	Spreads to the caller's own request path — order placement slows because Twilio is slow	Contained to that channel's worker pool; push/email unaffected, order-svc unaffected
Retry behavior under provider slowness	Every producer independently retries → retry storm that adds load exactly when the provider is struggling	Workers back off with jitter (section 13); queue backlog absorbs the slowdown instead of amplifying it
Burst handling (campaign spike)	Producers block or fail en masse; no buffer	Kafka absorbs the burst as a fast sequential append; workers drain at a sustainable rate
Crash mid-fan-out	Whatever hasn't been called yet is silently lost — no record it was even attempted	Message durably persisted in Kafka before any provider is called; redelivered and retried, never silently dropped
Operational cost	Fewer moving parts, easier to reason about <i>when everything is healthy</i>	Extra infra (Kafka, workers, DLQ) — but that infra is precisely what keeps failures from cascading

Rule of thumb: Any time a request's success depends on a third party you don't control, put a durable queue between "I accepted your intent" and "I called that third party." The queue is what turns "the provider is having a bad day" from an incident into a Tuesday.

16. Deep-dive: the crash-after-send problem — exactly-once vs. at-least-once

Interviewer: Walk me through this precisely. A worker pulls a message, calls SES, the email actually goes out — and *then* the worker crashes, before it acks/commits the message back to the queue. What happens, and how do you stop the user getting that same email twice?

Candidate: This is the sharpest edge case in the whole design, because it isn't really about notifications — it's the general problem of **combining an external side effect with our own bookkeeping in a way that isn't transactional**. You cannot atomically "call Twilio" and "record that I called Twilio" — those are two separate systems, and a crash between them is always possible. Let me show exactly where it bites.

Why it happens

The naive worker implementation records success *after* the side effect — that ordering feels natural ("only mark it done once it's actually done"), but it's exactly backwards for crash safety:

NAIVE WORKER (record-after-send):

```
t0 worker receives message #482 (order_shipped, user 9001, email)
t1 worker calls SES.send(...) → SES accepts, email is genuinely in flight to the user
t2 *** worker process crashes *** (OOM kill, deploy, node eviction – doesn't matter which
    - dedup key was never written
    - Kafka offset was never committed
-----
t3 Kafka's consumer-group rebalance / visibility timeout fires:
    message #482 was never acked → REDELIVERED to another worker
t4 new worker receives message #482 again – looks IDENTICAL to a fresh message
t5 dedup key doesn't exist (never written in t1-t2) → treated as "newly set", proceed
t6 worker calls SES.send(...) AGAIN → user 9001 gets the SAME email a second time
```

The crash landed in the one window where the side effect (t1) had already happened but our *record* of it (which would live at t2, right after) never got written. At-least-once delivery did exactly what it promises — delivered the message at least once — the bug is that our bookkeeping wasn't durable *before* the side effect, so the redelivery looked indistinguishable from a brand-new send.

Ranked fixes

1. Idempotency/dedup key per (user, notification), written *before* the send — not after. Flip the order from the naive worker: `SET dedup:{userId}:{notificationId} NX EX 24h` **first**, and only call SES if that succeeds. Now the redelivered message in t4 finds the key already present (written back in the original t1-equivalent step, *before* the provider call) and skips the second send entirely.

FIXED WORKER (record-before-send):

```
t0 worker receives message #482
t1 SET dedup:9001:notif_482 NX EX 24h → newly set, durable in Redis NOW
t2 worker calls SES.send(...) → SES accepts
t3 *** worker crashes before Kafka ack ***
-----
t4 message #482 REDELIVERED to another worker
t5 SET dedup:9001:notif_482 NX → already exists (from t1) → SKIP send
t6 mark delivery_status = duplicate-suppressed, done – user gets exactly ONE email
```

This closes the exact window from the naive version. It opens a *smaller*, different window instead: if the crash happens between t1 and t2 — key written, but SES was never actually called — the redelivered message will also see "already exists" and skip, meaning the notification is now **silently never sent**. That's a real tradeoff (duplicate risk traded for a much narrower drop risk), which is why a plain boolean dedup key isn't quite enough — see fix #1a.

1a. Make it a small state machine, not a boolean. Write `PENDING` before calling the provider, `SENT` after it succeeds. On redelivery: `SENT` → skip (done); `PENDING` and *fresh* (a few seconds old) → another worker may genuinely still be mid-call, skip and let it finish; `PENDING` and *stale* (older than a generous timeout, e.g. 60s — well past any realistic provider call) → safe to treat as abandoned and retry the send. This shrinks the ambiguous window from "forever" to "one timeout interval," at the cost of a rare, bounded double-send in that narrow stale-but-actually-in-flight sliver — which is where the next fixes come in.

2. Accept at-least-once + dedup as the practical answer — true exactly-once is impossible.

Exactly-once delivery *across an external system* would require an atomic transaction spanning Kafka's offset commit, our dedup store, and Twilio/SES's own send — three independent systems with no shared transaction coordinator. This is the same shape as the two-generals problem: no amount of protocol cleverness makes it disappear. The achievable, correct goal is **at-least-once delivery + idempotent handling on our side**, which is what fix #1/#1a actually is. Any time someone proposes "just make it exactly-once," the honest answer is "we can't — we can only make duplicates provably rare and harmless."

3. Provider-side idempotency keys, where available. Pass a client-generated idempotency key on the SES/Twilio call itself (many provider APIs accept one). Then even in the narrow window where we call the provider twice (the stale-PENDING retry case above), the *provider* collapses both calls into a single actual send. This is a second independent layer, not a replacement for

1 — it protects exactly the sliver that #1a can't fully close, and it's free if the provider

supports it.

4. Per-user rate limiting as a backstop, not a fix. None of the above is airtight against a *systemic* bug (a bad deploy that resets dedup state, a misconfigured retry loop). A per-user cap — `INCR rate: {userId}:{channel} EX 60`, reject/delay beyond N per minute — doesn't prevent a duplicate, but it bounds the blast radius: instead of a bug potentially causing an unbounded retry loop that spams one user hundreds of times, the damage is capped at N.

Recommended approach

Dedup key **written before the provider call**, upgraded to a small **state machine** (`PENDING` → `SENT`) with a stale-timeout for reconciling abandoned `PENDING` states, is the primary mechanism — it's the thing actually preventing duplicates in the common case. Layer **provider-side idempotency keys** on top wherever the provider supports them, since they close the one residual window (stale-PENDING retry) almost for free. Keep **per-user rate limiting** as the unconditional backstop for the case where dedup itself fails for a reason we didn't predict. And frame all of it, when asked, around the real constraint: we engineer for at-least-once + idempotency, because exactly-once across a queue *and* a third-party HTTP call is not on the menu.

What to say out loud: "You can't get exactly-once across our queue and an external provider — that would need one atomic transaction spanning Kafka, our dedup store, and Twilio, which don't share a transaction coordinator. So I engineer for at-least-once delivery plus an idempotency key

written before the provider is ever called, upgraded to a small PENDING/SENT state machine so a crash mid-call is reconciled by a timeout instead of blindly resent or silently dropped. Provider-side idempotency keys close the last sliver where our own retry might double-call the provider, and a per-user rate limit is the backstop that bounds the damage if any of that ever fails in a way I didn't anticipate."

Summary — the mental model

A notification system is fundamentally a **decoupling problem**: fast, bursty internal producers on one side, slow and occasionally-unreliable third-party providers on the other, and a hard requirement that the producer never waits on the provider. The queue (Kafka) is the backbone that makes that decoupling real — producers get a **202 Accepted** the instant their event is durably enqueued, and independent **per-channel worker pools** drain it at whatever rate each provider can sustain. Because queues only give **at-least-once** delivery, an atomic Redis **dedup key per (user, event, channel)** is what actually prevents double-notifying a user — that's the correctness backbone of the whole design. Preferences and templates are cached in front of a small relational store; the massive delivery log lives in a wide-column/KV store sized for its write volume. Failures are handled by **exponential backoff with jitter** (never a retry storm), a **DLQ** instead of a silent drop, and strict **priority separation** so one marketing campaign can never delay a security alert.